

校园跑腿 Web 平台

软件设计规格说明书

项目名称：校园跑腿 Web 平台

编制日期：2026-06-12

版 本：V4.0

1、引言

1.1 编写目的

软件设计过程包括体系结构设计、数据设计和部署设计，分别从不同的层次、不同的视角对软件系统进行了设计，产生了不同的软件制品。在完成上述设计工作之后，需要将这些软件设计成果进行整合，形成一个系统、完整的软件设计方案。本文档以软件设计规格说明书的形式描述了该设计方案，方便评审人员对设计方案的正确性、合理性等方面进行评审。

1.2 读者对象

用户、软件设计人员、程序员、软件需求分析人员、质量保证人员、软件测试工程师、配置管理工程师。

1.3 软件项目概述

项目名称：校园跑腿 Web 平台

用户：武汉大学在校学生

开发单位：校园跑腿系统开发小组（6 人）

软件项目的背景和大致功能：

随着高校校园生活节奏的加快，学生日常生活中经常出现找人帮忙取快递、带外卖、代买东西等临时性需求。目前校内缺乏一个统一、便捷的跑腿服务平台，需求发布和接单往往依赖 QQ 群、微信群等零散方式，信息杂乱、效率低下。

本软件专为解决校园跑腿服务的管理和匹配问题而设计，为用户提供一个完整的跑腿服务线上平台。主要功能包括：用户注册与认证，支持跑腿和接单双模式切换；四种跑腿订单类型（外卖、快递、代买、自定义）；订单全生命周期管理（发布、接单、配送、结算、评价）；账户余额管理与交易流水记录。

1.4 文档概述

1 软件设计约束部分。它包括软件设计目标和原则、软件设计受到的约束和限制。

2 软件设计部分。它主要分为软件体系结构设计、数据设计以及部署设计。

1.5 定义

术语	说明
FastAPI	Python 现代高性能 Web 框架，基于 ASGI
SQLAlchemy	Python ORM 框架
ORM	对象关系映射

1.6 参考资料

1. 《软件工程》教育部"101 计划"教材，清华大学出版社，第 6 章
2. GB 8567-1988，《计算机软件产品开发文件编制指南》
3. 《校园跑腿 Web 平台软件需求说明书》，本小组

4. 软件设计约束

2.1 软件设计目标和原则

软件设计的目标是，根据软件系统的需求，综合考虑软件开发过程中的各种制约因素，遵循软件工程的设计原则，给出软件系统的实现解决方案和蓝图，产生可指导编码实现的设计模型及文档。

模块化与高内聚低耦合的原则。系统划分为 6 个独立模块，每个模块功能清晰、边界明确。模块内部高度内聚，模块间通过共享数据库耦合。

单一职责的原则。每个模块只负责一类业务，不交叉。

开闭的原则。可新增跑腿类型而不修改已有模块代码。

抽象和逐步求精的原则。从概要设计到详细设计逐步细化，直至可指导编码实现。

2.2 软件设计的约束和限制

运行环境要求：Python 3.10+、MySQL 8.0、主流浏览器

开发语言：Python

Web 框架：FastAPI

ORM 框架：SQLAlchemy

标准规范：PEP 8 编码风格

开发工具：VS Code、MySQL Workbench

部署方式：华为云 ECS 弹性云服务器 2 核 4GB 公网 IP

5. 软件设计

3.1 软件体系结构设计

该软件系统在逻辑上划分为四层：表现层、控制层、业务逻辑层和数据访问层，见图 3.1.1。

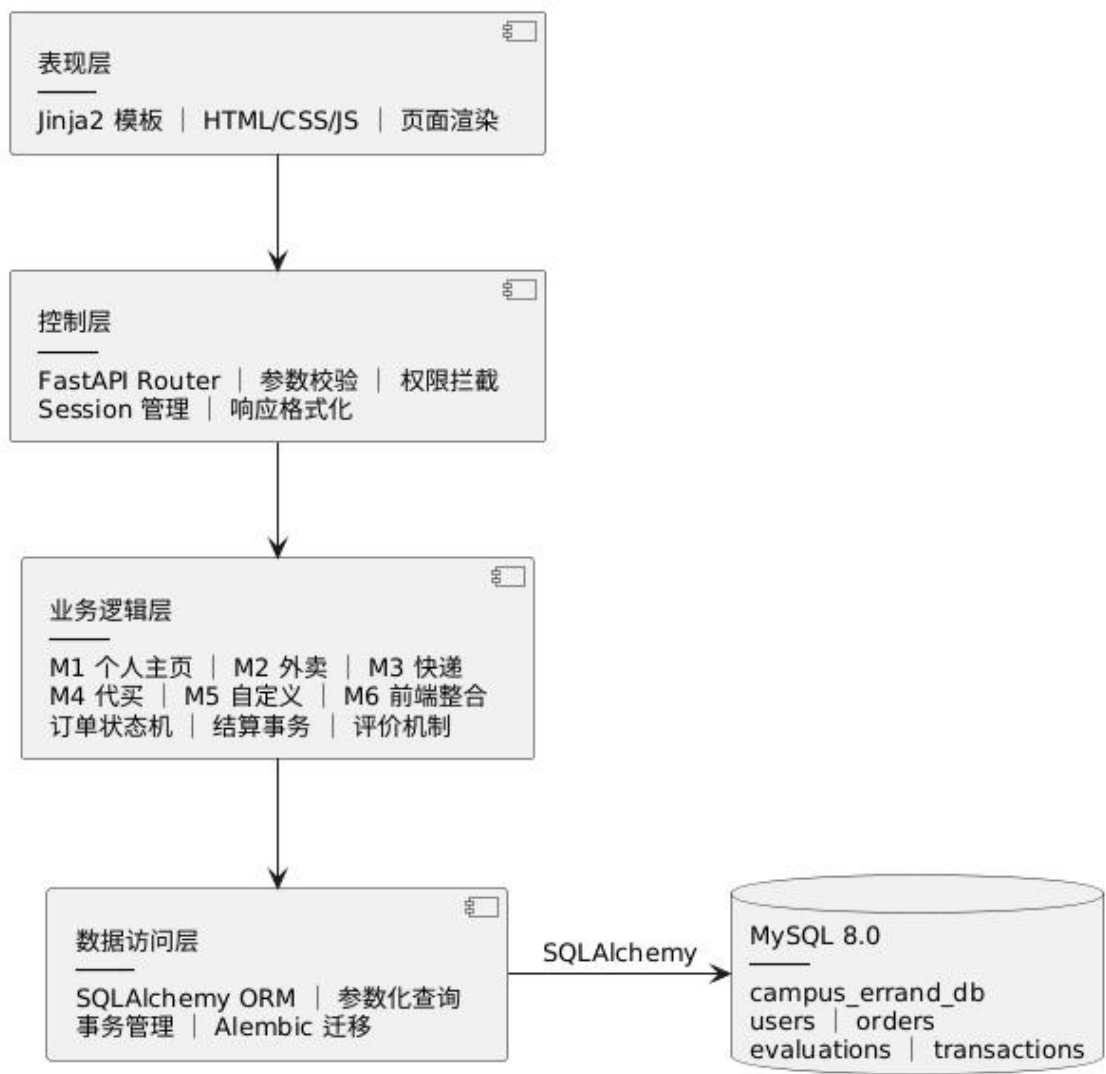


图 3.1.1 校园跑腿 Web 平台的软件体系结构图

表现层负责用户界面渲染和交互响应，采用 Jinja2 模板引擎配合 HTML/CSS/JS 技术实现。

控制层负责请求分发、参数校验和权限控制，基于 FastAPI Router 实现，利用 FastAPI 的依赖注入机制实现 Session 管理和登录拦截。

业务逻辑层负责执行业务规则、订单状态机管理和结算事务处理，包含 6 个业务模块，每个模块通过 FastAPI Router 独立注册。

数据访问层负责 ORM 数据读写和事务管理，基于 SQLAlchemy 实现，利用 FastAPI 异步特性提升数据库操作性能。

编号	构件名称	负责人	供给接口
P1	个人主页模块	成员 1	注册、登录、个人信息、模式切换
P2	外卖跑腿模块	成员 2	外卖订单发布、接单、配送、结算
P3	快递跑腿模块	成员 3	快递订单发布、接单、配送、结算
P4	代买服务模块	成员 4	代买订单发布、接单、配送、结算
P5	自定义跑腿模块	成员 5	自定义订单发布、接单、配送、结算
P6	前端整合模块	成员 6	首页导航、基础模板、公共样式

3.2 数据设计

3.2.1 概念结构设计

该软件系统经过 E-R 分析，识别出五个核心实体：用户、骑手、订单、评价和交易流水，E-R 图见图 3.2.1。

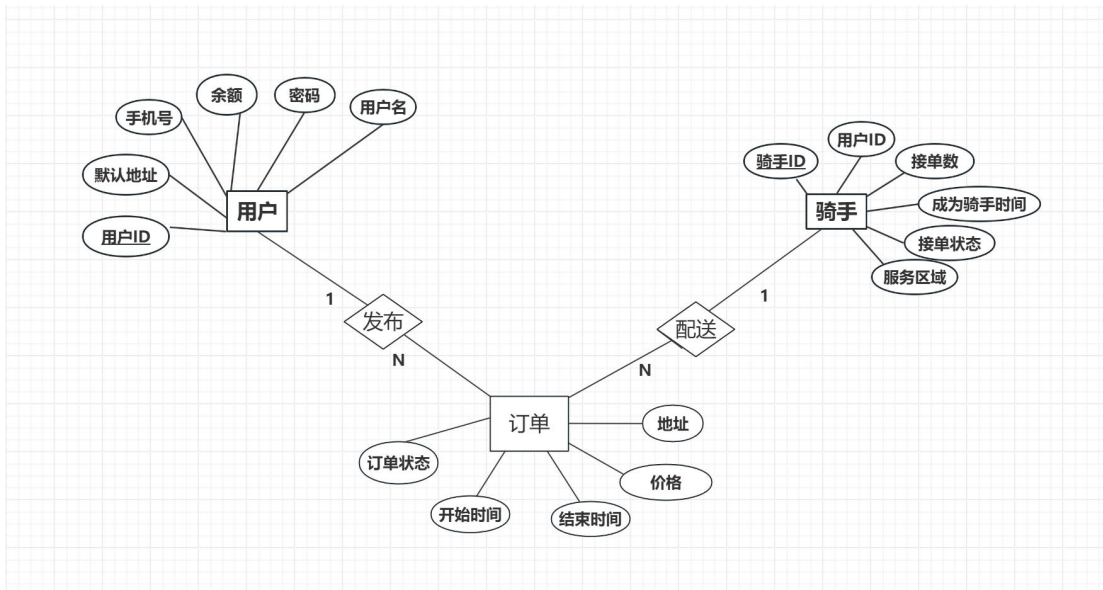


图 3.2.1 校园跑腿 Web 平台 E-R 图

用户是系统的核心账号实体。骑手是独立实体，用户注册为骑手后在 `riders` 表中创建对应记录。订单是业务核心实体，用户可发布多个订单（1:n），骑手可配送多个订单（1:n）。

3.2.2 数据库表设计

依据 E-R 图，设计 5 张数据库表。

用户表（users）

字段名	类型	约束	说明
user_id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	用户 ID
username	VARCHAR(64)	UNIQUE, NOT NULL	用户名
password_hash	VARCHAR(256)	NOT NULL	bcrypt 密码哈希
phone	VARCHAR(20)	UNIQUE, NOT NULL	手机号
balance	DECIMAL(10,2)	NOT NULL, DEFAULT 0	账户余额
default_address	VARCHAR(256)	DEFAULT NULL	默认地址
created_at	DATETIME	NOT NULL	注册时间

骑手表（riders）

字段名	类型	约束	说明
rider_id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	骑手 ID
user_id	BIGINT UNSIGNED	FK → users, UNIQUE	关联用户 ID (唯一)
credit_score	DECIMAL(3,1)	DEFAULT NULL	信用评分
total_orders	INT UNSIGNED	DEFAULT 0	总接单数
praise_rate	DECIMAL(3,2)	DEFAULT NULL	好评率
service_area	VARCHAR(128)	DEFAULT NULL	服务区域
rider_status	VARCHAR(20)	DEFAULT offline	接单状态
created_at	DATETIME	NOT NULL	成为骑手时间

订单表 (orders)

字段名	类型	约束	说明
order_id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	订单 ID
order_no	VARCHAR(32)	UNIQUE, NOT NULL	订单编号
order_type	ENUM	NOT NULL	订单类型
publisher_id	BIGINT UNSIGNED	FK → users	发布者 ID
rider_id	BIGINT UNSIGNED	FK → riders	接单骑手 ID
status	ENUM	NOT NULL, DEFAULT pending	订单状态
reward	DECIMAL(10,2)	NOT NULL	酬劳金额
delivery_sub_status	VARCHAR(20)	DEFAULT NULL	配送子状态
delivery_addr	VARCHAR(256)	NOT NULL	配送地址
biz_fields	JSON	NOT NULL	业务扩展字段
created_at	DATETIME	NOT NULL	创建时间
completed_at	DATETIME	DEFAULT NULL	完成时间

3.2.3 数据操作接口

UserManager:

register(username, phone, password) 注册

login(account, password)

get_profile(user_id)

become_rider(user_id, service_area)

update_balance(user_id, amount)

OrderManager:

create(publisher_id, order_type, biz_fields, reward, delivery_addr)

get_pending(order_type)

accept(order_id, rider_id)

update_status(order_id, status)

cancel(order_id)

complete(order_id)

结算事务：校验 delivered → 更新 completed → 发布者扣款 + 流水 → 骑手入账 + 流水 → COMMIT。

登录

获取用户信息

注册为骑手

更新余额

发布

待接单列表

接单

更新配送状态

取消订单

确认收货并结算

3.3 部署设计

校园跑腿 Web 平台采用集中式部署，部署于华为云 ECS 弹性云服务器。Web 服务器、应用服务器和数据库服务器部署在同一台云服务器上，基于 FastAPI（Uvicorn）运行，MySQL 8.0 存储数据。

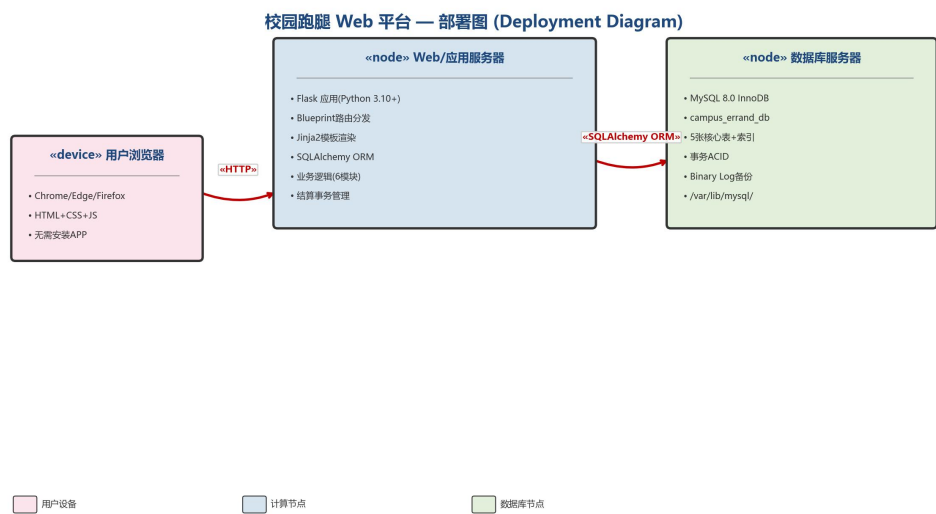


图 3.3.1 校园跑腿 Web 平台的部署图

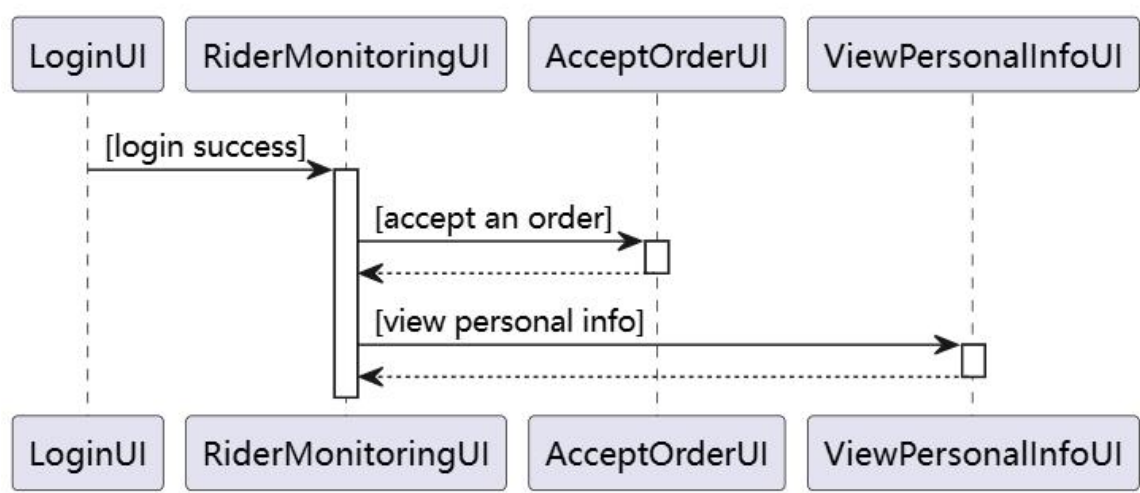
节点	构造型	配置	软件栈
----	-----	----	-----

用户浏览器	device	无特殊要求	Chrome/Edge/Firefox
华为云 ECS 服务器	node	2 核 CPU 4GB 内存 40GB SSD 公网 IP	Python 3.10 + FastAPI + Uvicorn + MySQL 8.0

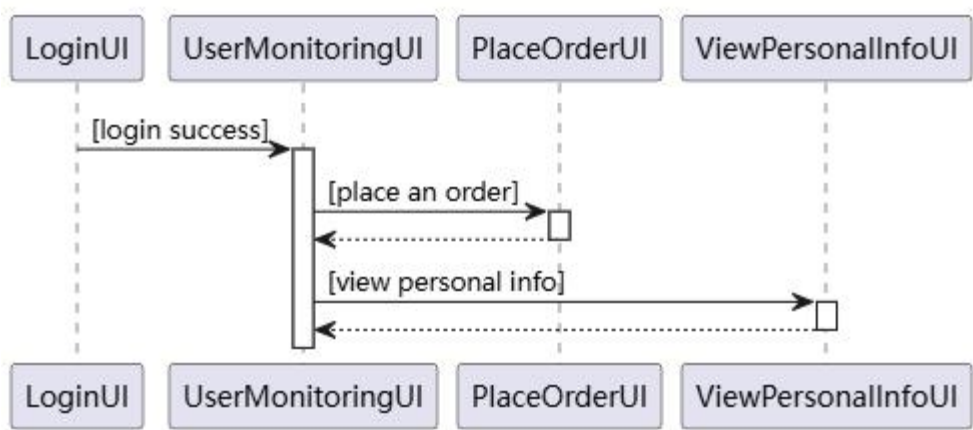
部署步骤：购买华为云 ECS（2 核 4GB，公网 IP）、安装 Python + MySQL + 依赖包、创建 campus_errand_db 并执行 Alembic 建表、配置安全组开放 80 端口、uvicorn 启动应用、浏览器通过公网 IP 访问。

UI 设计：

骑手端UI时序图



用户端UI时序图



— 确认接单

订单详情

类型 取快递
标题 中通快递取件
取件地址 菜鸟驿站5号柜
送达地址 3号宿舍楼501
下单用户 张三

酬劳 ¥5.00

接单后请按时完成配送。如遇到问题请及时联系用户。

返回

确认接单

跑腿系统

登录您的账号

用户名

请输入用户名

密码

请输入密码

角色

请选择角色

登录

原型页面 · LoginUI

新建订单

订单类型

请选择类型

订单标题

如：取中通快递

取件/购买地址

如：菜鸟驿站

送达地址

如：3号宿舍楼501

备注

补充说明.....

预估费用：¥5.00

提交订单

骑手接单

个人信息 退出

待接单 (共3单)

取快递 - 中通快递

菜鸟驿站5号柜 → 3号宿舍楼
距离约 200m

¥5.00

接单

代送文件

图书馆 → 行政楼302
距离约 350m

¥6.00

接单

代买 - 食堂午餐

二食堂 → 图书馆3楼
距离约 150m

¥8.00

接单

进行中的订单

- 取快递 - 中通快递

取件地址：菜鸟驿站5号柜 | 送至：3号宿舍楼

¥5.00

配送中
- 代买 - 食堂午餐

备注：二荤一素，不要辣 | 送至：图书馆3楼

¥8.00

待接单

已完成订单

- 取快递 - 圆通快递

完成时间：2026-06-11 15:30

¥5.00

已完成
- 代买 - 超市零食

完成时间：2026-06-10 20:15

¥6.00

已完成

+ 新建订单

个人信息

张

张三

普通用户

基本资料

用户ID

138****6789

U001

手机号

3号宿舍楼501

常用地址

2026-03-01

注册时间

订单统计

12

总订单

1

进行中

11

已完成

骑手信息

李

李四

认证骑手

★ 4.8

基本资料

骑手ID

139****1234

R001

手机号

校内

接单范围

2026-02-15

注册时间

接单统计

56

总接单

52

已完成

¥328

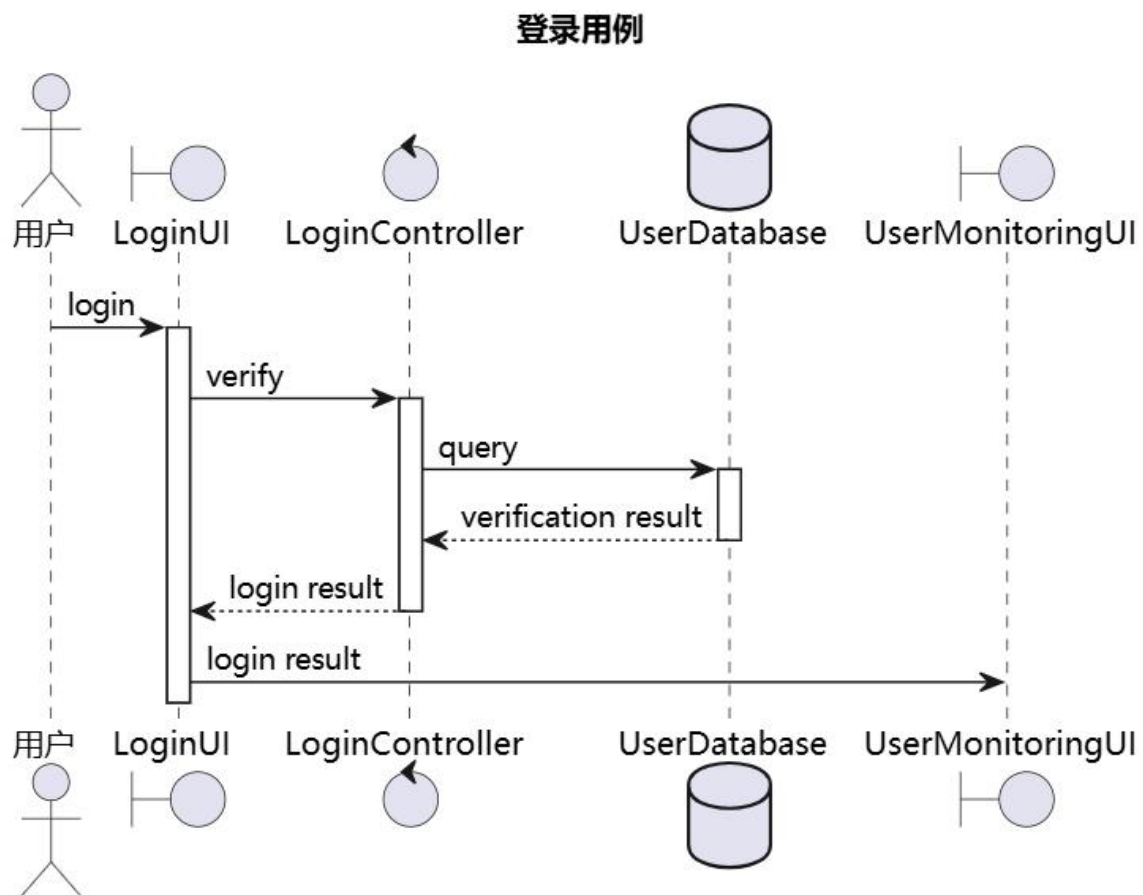
累计收入

- 登录页"LoginUI" : 职责是接收用户输入的用户名和密码以登录到系统之中
- 用户页“UserMonitoringUI” : 职责是查看未完成订单和已完成订单
- 用户下单页“PlaceOrderUI” : 职责是接收用户输入的订单信息并提交订单
- 骑手页“RiderMonitoringUI” : 职责是查看待接单的订单并选择接单

- 跑腿接单页“AcceptOrderUI”：职责是确认接单
- 用户查看个人信息页“ViewPersonalInfoUI”：职责是查看用户个人信息
- 骑手查看个人信息页“ViewRiderPersonalInfoUI”：职责是查看骑手个人信息

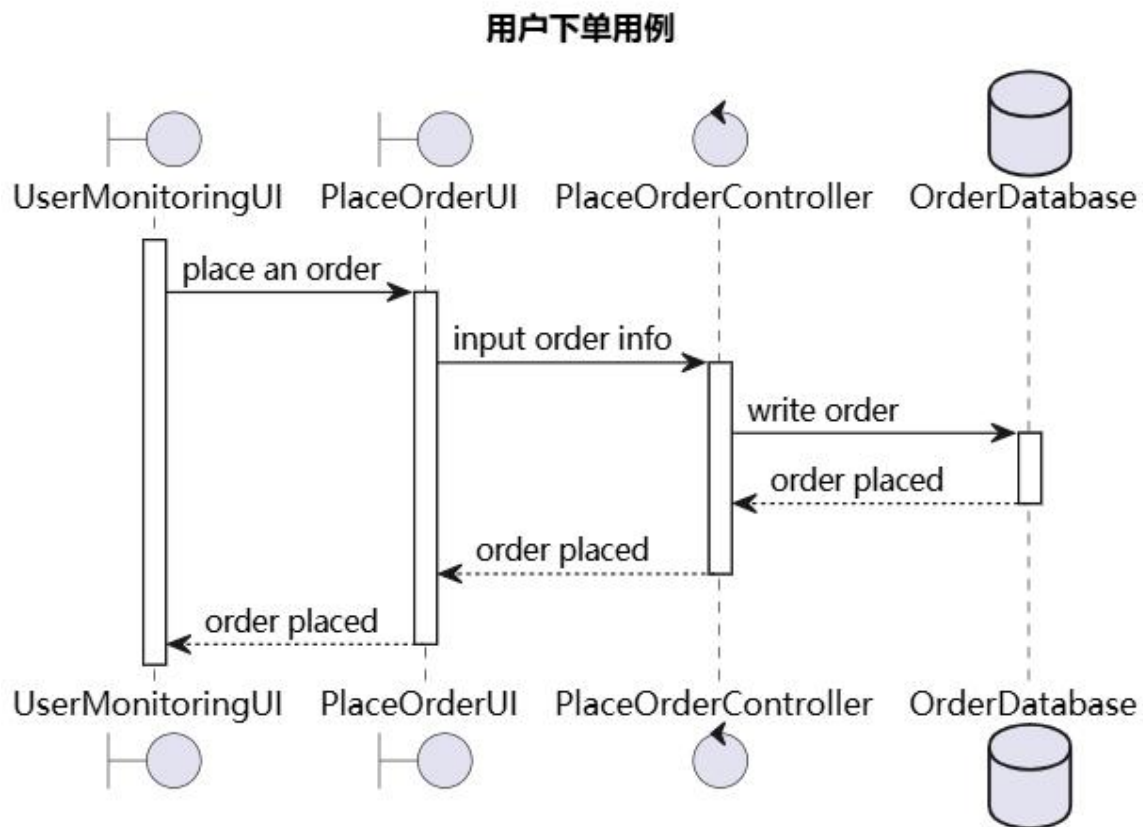
用例设计

登录用例



1. 用户在 **LoginUI** 输入用户名、密码并选择角色后触发登录；
2. LoginUI 调用 **LoginController** 进行身份验证；

3. LoginController 查询 **UserDatabase**，匹配用户名和密码；
4. UserDatabase 返回验证结果；
5. LoginController 返回登录结果给 LoginUI；
6. 登录成功后，LoginUI 跳转至 **UserMonitoringUI**（用户主页）。

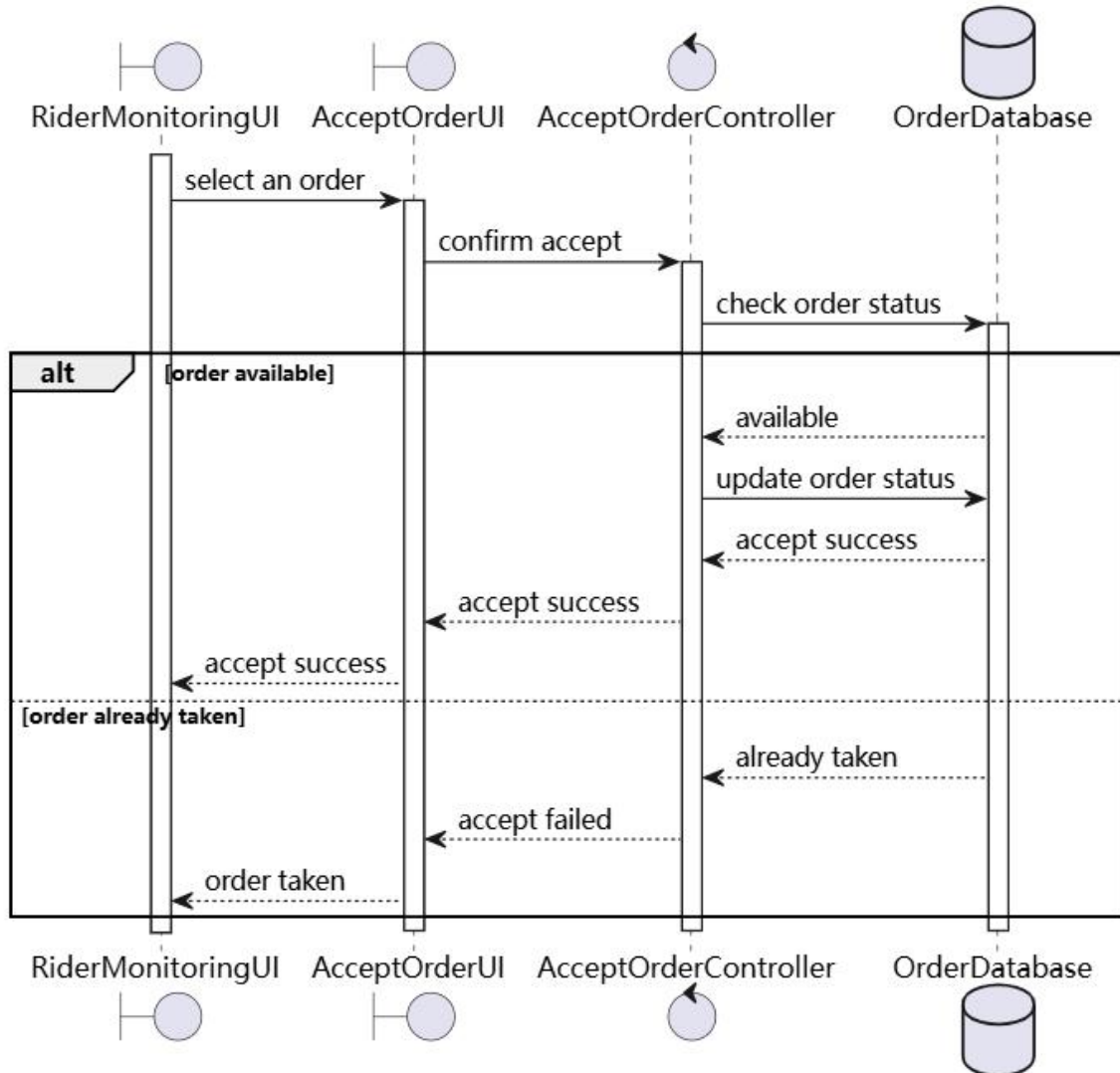


用户下单用例

1. 用户在 **UserMonitoringUI** 点击"新建订单"，跳转至 **PlaceOrderUI**；
2. 用户填写订单信息（类型、标题、取件地址、送达地址等）后提交；
3. PlaceOrderUI 将订单信息传递给 **PlaceOrderController**；
4. PlaceOrderController 将订单数据写入 **OrderDatabase**；

5. OrderDatabase 返回写入成功;
6. PlaceOrderController 返回提交结果给 PlaceOrderUI;
7. PlaceOrderUI 返回至 UserMonitoringUI, 订单创建完成。

骑手接单用例



骑手接单用例

1. 骑手在 **RiderMonitoringUI** 选择一个待接单订单, 跳转至 **AcceptOrderUI**;
2. AcceptOrderUI 调用 **AcceptOrderController** 发起接单确认;

3. AcceptOrderController 查询 **OrderDatabase** 检查订单状态:

- 若订单状态为 "**pending**" (可接单): 更新订单状态为"已接单", 返回接单成功;

- 若订单状态为 "**已接单**" (已被抢): 返回接单失败, 提示订单已被人接走;

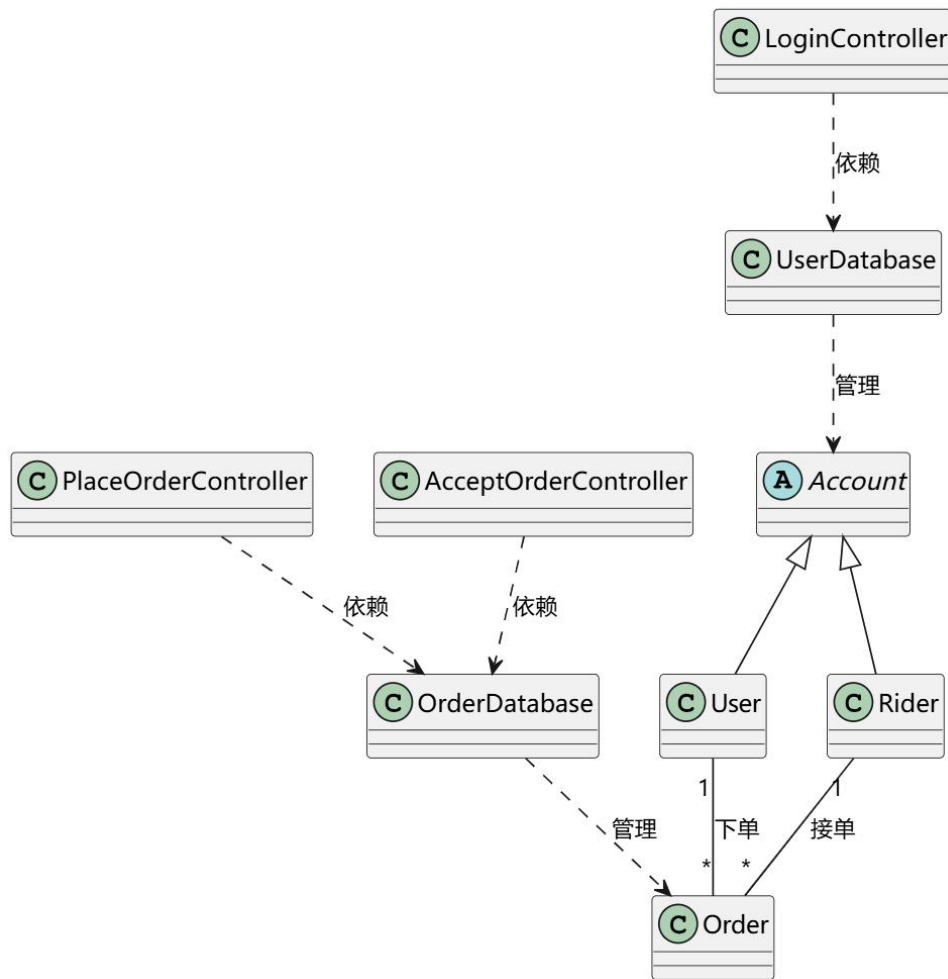
4. AcceptOrderController 将结果返回给 AcceptOrderUI;

5. AcceptOrderUI 返回结果给 RiderMonitoringUI, 刷新订单列表。

类设计

类概要设计

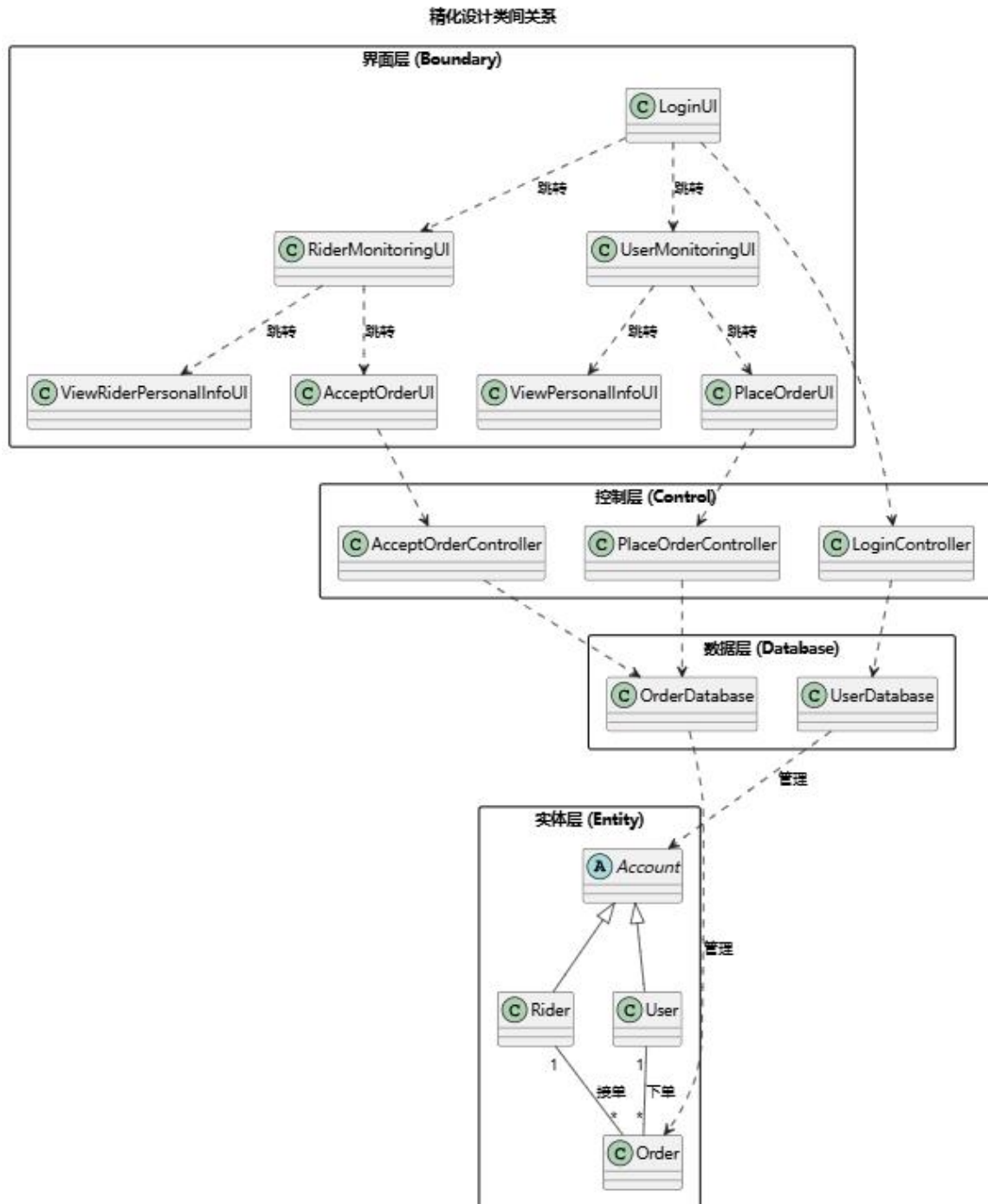
类概要设计



系统核心类分为四层：实体层（`Account` 抽象类派生出 `User` 和 `Rider`，以及 `Order`）、数据访问层（`UserDatabase`、`OrderDatabase`）、控制层（`LoginController`、`PlaceOrderController`、`AcceptOrderController`）。

- `User` 与 `Order` 为 **1 对多** 关联（一个用户下多个订单）；
- `Rider` 与 `Order` 为 **1 对多** 关联（一个骑手接多个订单）；
- 各 `Controller` 依赖对应的 `Database` 完成业务逻辑；
- 各 `Database` 管理对应的实体类。

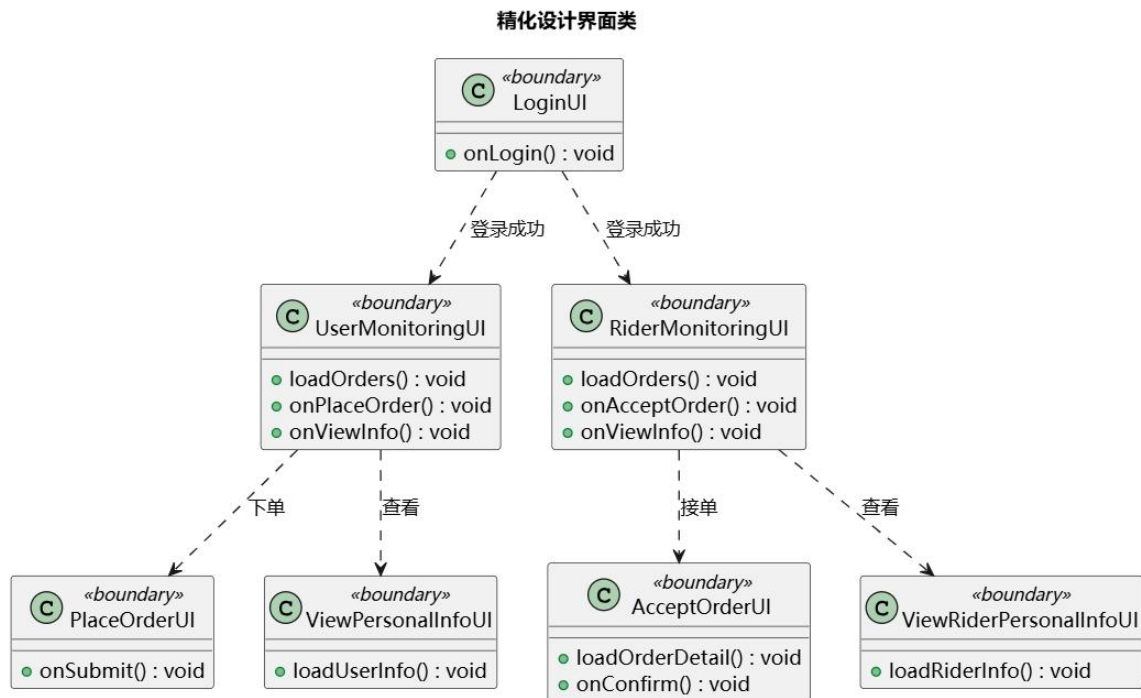
精化设计类间关系



将系统类按三层架构分包组织：

界面层通过虚线依赖控制层，控制层通过虚线依赖数据层，数据层通过虚线管理实体。User 和 Rider 继承自 Account。界面间通过虚线表示跳转关系。

精化设计界面类



七个界面类及核心方法：

| 界面类 | 关键方法 |

|-----|-----|

| LoginUI | onLogin() |

| UserMonitoringUI | loadOrders()、onPlaceOrder()、onViewInfo() |

| PlaceOrderUI | onSubmit() |

| RiderMonitoringUI | loadOrders()、onAcceptOrder()、onViewInfo() |

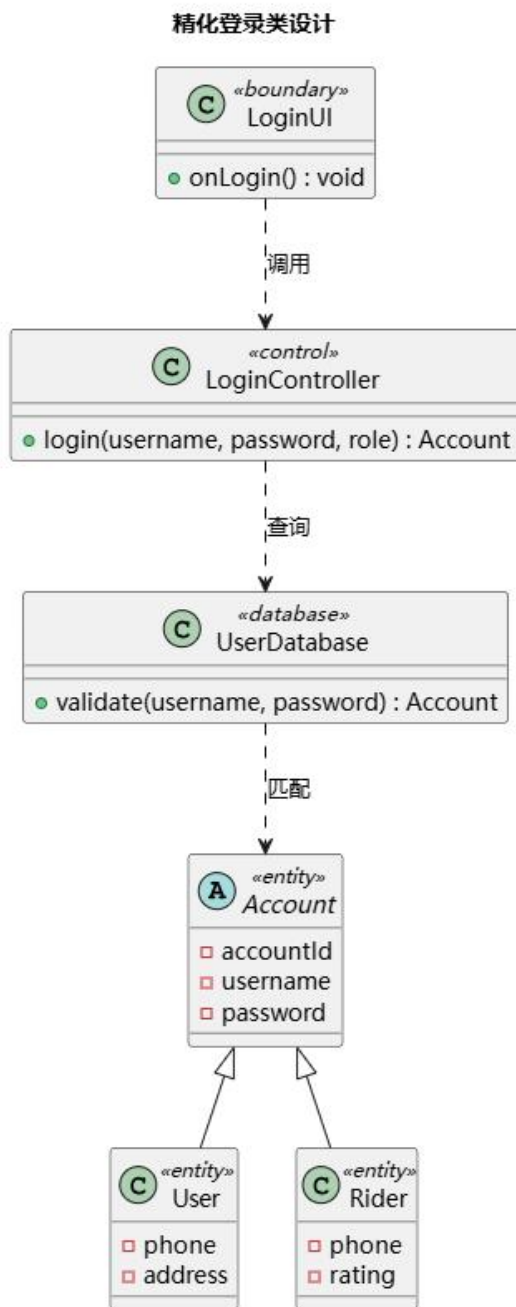
| AcceptOrderUI | loadOrderDetail()、onConfirm() |

| ViewPersonalInfoUI | loadUserInfo() |

| ViewRiderPersonalInfoUI | loadRiderInfo() |

界面间跳转：LoginUI 可跳转至 UserMonitoringUI 或 RiderMonitoringUI；用户端可跳转至下单页与个人信息页；骑手端可跳转至确认接单页与个人信息页。

精化登录类设计

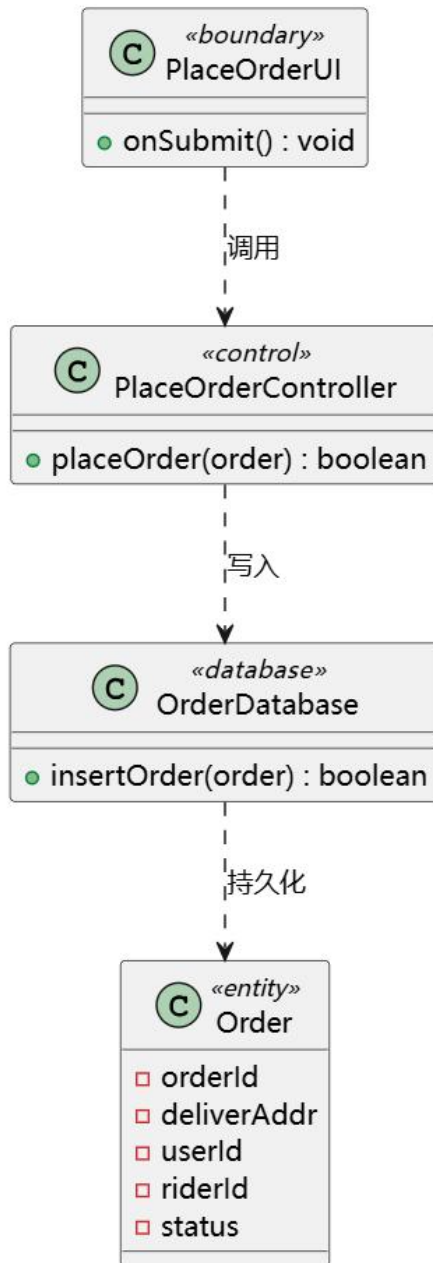


- **LoginUI** 接收用户输入，调用 LoginController;

- **LoginController** 负责身份验证，依赖 UserDatabase 查询；
- **UserDatabase** 匹配 Account 实体，返回 User 或 Rider 子类；
- **Account** 为抽象实体，包含公共字段（accountId、username、password），User 和 Rider 分别继承并扩展自有属性。

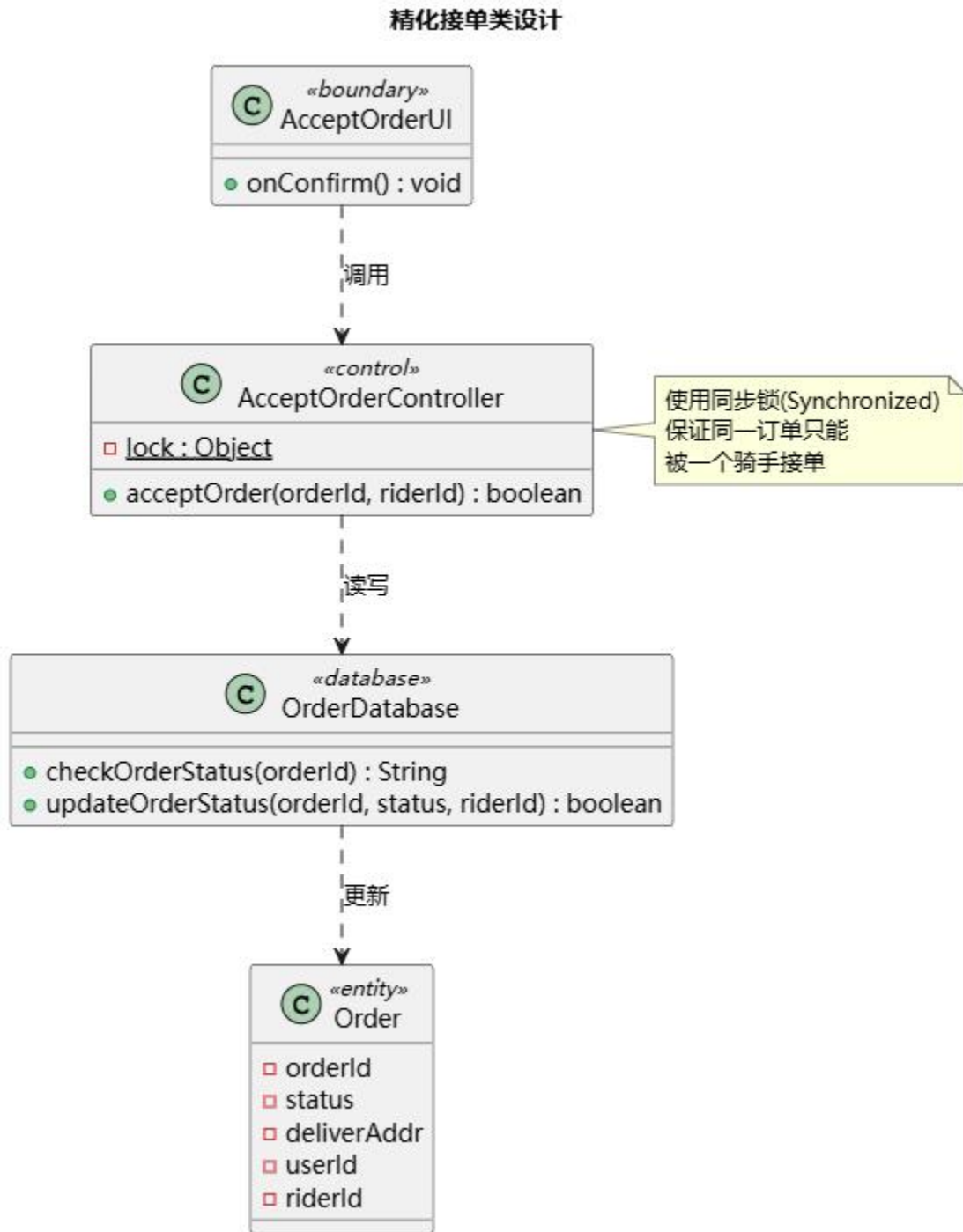
精化下单类设计

精化下单类设计



- **PlaceOrderUI** 接收订单表单，提交后调用 **PlaceOrderController**;
- **PlaceOrderController** 将订单数据写入 **OrderDatabase**;
- **OrderDatabase** 持久化 **Order** 实体（包含 `orderId`、`deliverAddr`、`userId`、`riderId`、`status` 等字段）。

精化接单类设计

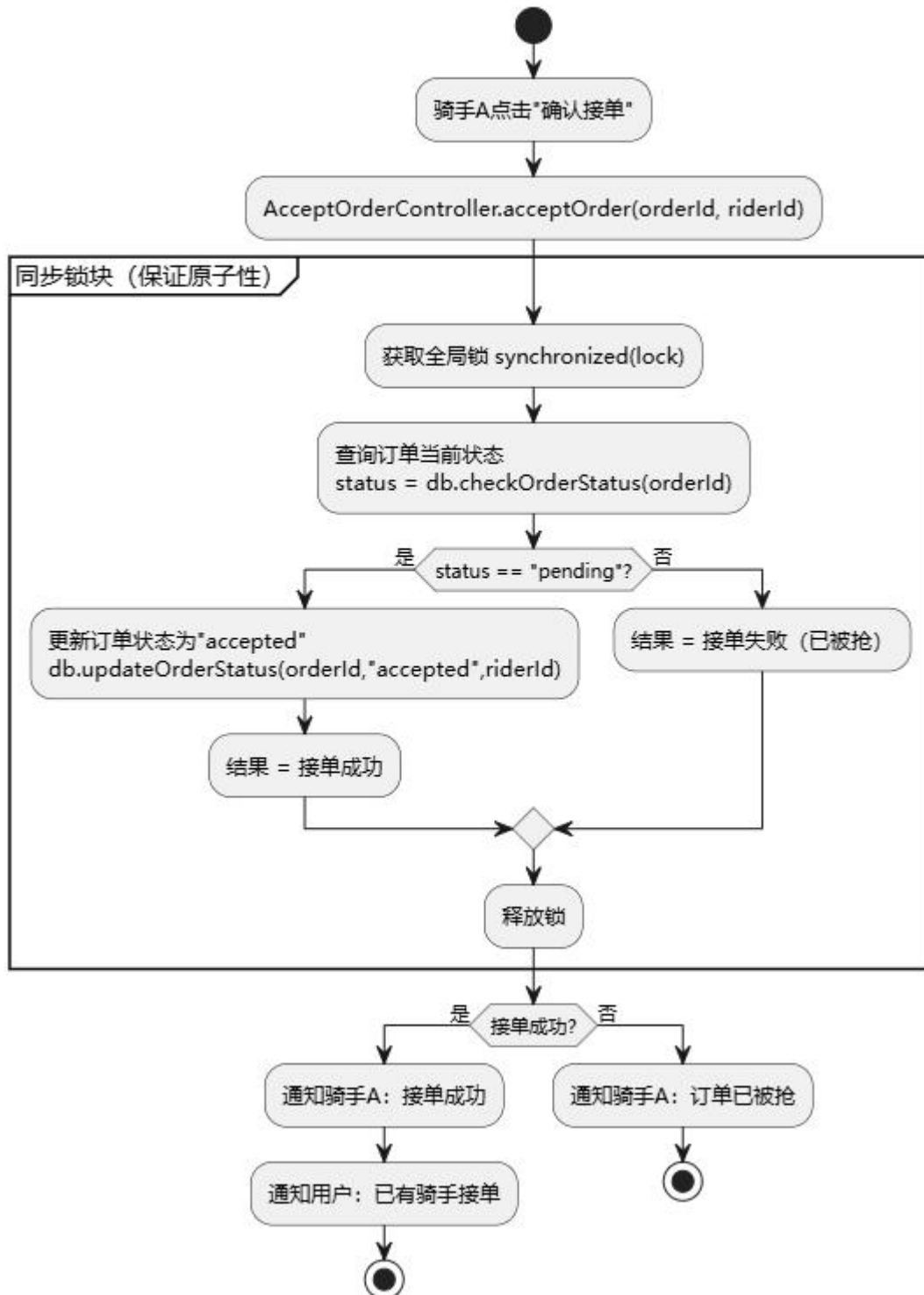


- **AcceptOrderUI** 展示订单详情，骑手确认后调用 `AcceptOrderController`;
- **AcceptOrderController** 持有静态同步锁 `'lock'`，通过 `checkOrderStatus()` 检查订单状态后调用 `updateOrderStatus()` 更新，保证并发安全;

- **OrderDatabase** 提供状态查询与更新接口，操作 Order 实体。

骑手接单算法（多骑手并发抢单）

骑手接单算法流程图（多骑手并发抢单）



核心流程：

1. 骑手点击"确认接单"，`AcceptOrderController.acceptOrder()` 被调用；

2. 进入同步锁块（synchronized），获取全局锁；

3. 查询订单当前状态：

- 若为 **pending**：更新状态为 accepted，接单成功，通知用户；

- 若已被抢：接单失败，提示骑手订单已被人接走；

4. 释放锁。

通过 synchronized 锁保证同一订单在同一时刻仅被一个骑手成功接单。